

Documento de Resultados

Proyecto: Alineamiento de Secuencias de ADN
Programación Paralela y Concurrente | Julio 2026

Arquitectura general

El flujo conserva una única generación de datos para comparar todas las estrategias con la misma secuencia de ADN y el mismo conjunto de patrones.

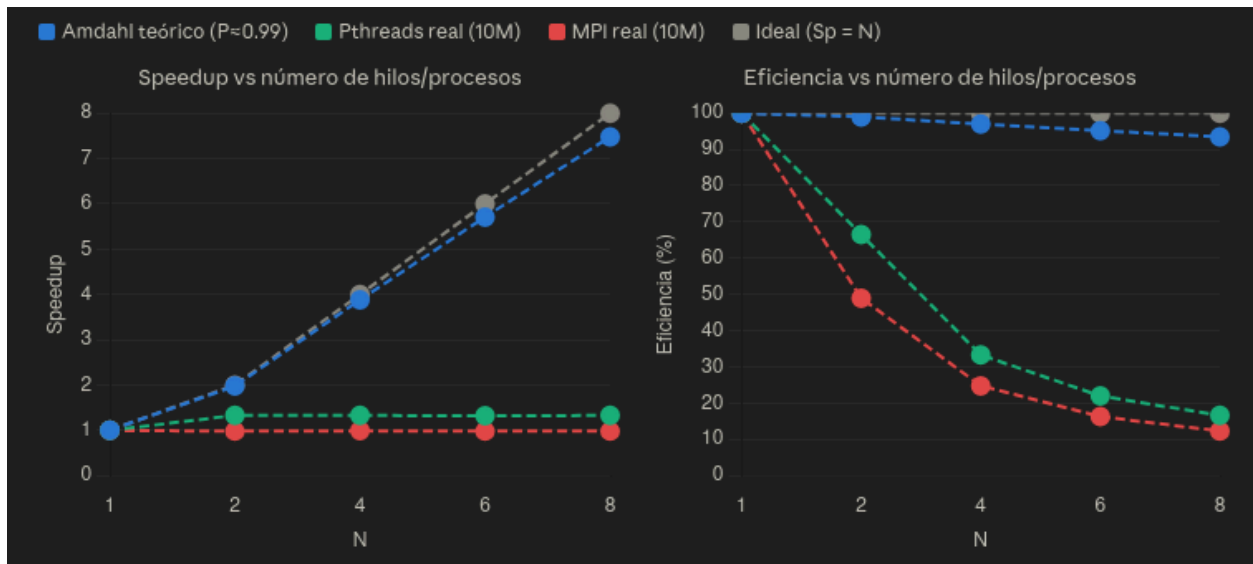
Etapa	Responsabilidad	Salida
Entrada y parámetros	Leer longitud de ADN, cantidad de patrones, longitud de patrón y número de hilos.	Estructura parametros_t.
Generación de datos	Crear secuencia de ADN y patrones aleatorios con los nucleótidos A, T, C y G.	cadena_adn y arreglo patron_t.
Referencia secuencial	Ejecutar fuerza bruta de un patrón contra todas las posiciones.	Primera posición encontrada o NO_ENCONTRADO.
Pthreads optimizado	Repartir dinámicamente patrones entre un pool persistente de hilos.	Resultados equivalentes a la versión secuencial.
OpenCL	Ejecutar work-items por patrón y posición; conservar la primera coincidencia.	Resultados copiados desde buffers OpenCL.
MPI	Distribuir patrones entre procesos y recolectar resultados en el proceso root.	Resultados globales para impresión final.

Módulos principales

Modulo	Función principal
cadenas.c/ cadenas.h	Reserva memoria, genera ADN, genera patrones y define patron_t
parametros.c / parametros.h	Procesa argumentos de línea de comandos y valores por defecto.
busqueda_lineal.c	Implementa la búsqueda secuencial base por fuerza bruta.
busqueda_pthread.c	Implementa el pool persistente de hilos y asignación dinámica de patrones.
busqueda_opencl.c / .cl	Prepara buffers, compila el kernel y ejecuta la búsqueda paralela OpenCL.
busqueda_MPI.c	Distribuye patrones entre procesos MPI y recolecta resultados.
tests/test.c	Válida casos determinísticos y compara versiones contra la secuencial.

Tablas de Resultados

Tabla de Metricas (10M, 5000, 1000)					
	Tiempo Seq. (s)	Tiempo Par. (s)	Speedup Real	Speedup Amdahl	Eficiencia
Secuencial	143.71	143.71	1	1	1
Pthreads 2H	143.71	108.04	1.33	1.98	0.67
Pthreads 4H	143.71	107.7	1.33	3.88	0.33
Pthreads 6H	143.71	108.77	1.32	5.71	0.22
Pthreads 8H	143.71	107.89	1.33	7.48	0.17
MPI 2H	143.71	146.09	0.98	1.98	0.49
MPI 4H	143.71	145.8	0.99	3.88	0.25
MPI 6H	143.71	146.43	0.98	5.71	0.16
MPI 8H	143.71	146.36	0.98	7.48	0.12
OpenCL	143.71	3.65	39.37	n/a	n/a
Tabla de Metricas (5M, 2500, 500)					
	Tiempo Seq. (s)	Tiempo Par. (s)	Speedup Real	Speedup Amdahl	Eficiencia
Secuencial	36.03	36.03	1	1	1
Pthreads 2H	36.03	26.68	1.35	1.98	0.68
Pthreads 4H	36.03	26.6	1.35	3.88	0.34
Pthreads 6H	36.03	26.94	1.34	5.71	0.22
Pthreads 8H	36.03	27.06	1.33	7.48	0.17
MPI 2H	36.03	36.49	0.99	1.98	0.5
MPI 4H	36.03	36.45	0.99	3.88	0.25
MPI 6H	36.03	36.54	0.99	5.71	0.17
MPI 8H	36.03	36.91	0.98	7.48	0.12
OpenCL	36.03	1.01	35.67	n/a	n/a
Tabla de Metricas (1M, 500, 100)					
	Tiempo Seq. (s)	Tiempo Par. (s)	Speedup Real	Speedup Amdahl	Eficiencia
Secuencial	1.45	1.45	1	1	1
Pthreads 2H	1.45	1.07	1.36	1.98	0.68
Pthreads 4H	1.45	1.08	1.34	3.88	0.34
Pthreads 6H	1.45	1.07	1.36	5.71	0.23
Pthreads 8H	1.45	1.06	1.37	7.48	0.17
MPI 2H	1.45	1.71	0.85	1.98	0.43
MPI 4H	1.45	1.72	0.84	3.88	0.21
MPI 6H	1.45	1.72	0.84	5.71	0.14
MPI 8H	1.45	1.71	0.85	7.48	0.11
OpenCL	1.45	0.24	6.04	n/a	n/a



Discusión de Diferencias

La diferencia entre el speedup real y el speedup teórico de Amdahl

En las tres pruebas (10M, 5M y 1M pares de bases), la fracción paralelizable se mantuvo alrededor de 0,99, ya que buscar patrones es mucho más costoso que generar los datos. Con ese valor, Amdahl predice un speedup de hasta 7,48x con 8 hilos/procesos. Sin embargo, el speedup real nunca superó 1,37x con Pthreads y siempre fue menor a 1,0x con MPI.

Esto confirma una limitación conocida del modelo: Amdahl asume que dividir el trabajo no tiene costo adicional, pero en la práctica cada hilo o proceso implica sincronización, comunicación y contención de recursos que el modelo ignora. En nuestros datos, esos costos superaron por completo la ganancia teórica esperada.

Pthreads: limitado por sincronización y entorno

El speedup se mantuvo constante en $\sim 1,33x$ sin importar el número de hilos (2, 4, 6 u 8), señal de un techo de rendimiento alcanzado muy temprano, por dos razones:

1. **Contención de caché:** los resultados se escriben en un arreglo compartido; hilos distintos escribiendo en posiciones cercanas generan invalidación cruzada de caché entre núcleos.
2. **Entorno virtualizado:** las pruebas corrieron en una VM, sin acceso exclusivo garantizado a los núcleos físicos. Esto se notó incluso en la versión secuencial, con variación de hasta

10% entre corridas idénticas.

La eficiencia cae de 67% (2 hilos) a 17% (8 hilos), confirmando que agregar hilos no ayuda más allá de cierto punto — máxime con 8 hilos en una máquina de 6 núcleos físicos, donde ya hay sobre-suscripción.

MPI: el costo de comunicación supera la ganancia

El speedup real fue menor a 1,0x en casi todas las configuraciones: la versión paralela fue más lenta que la secuencial. La causa es el diseño de comunicación — el proceso principal envía la cadena de ADN y los patrones a cada proceso y luego recolecta los resultados, un costo fijo que no se diluye al aumentar procesos. El caso más extremo fue con la entrada más chica (1M bp), donde el speedup cayó a 0,84x, porque ahí el costo de comunicación pesa proporcionalmente más frente al cómputo.

OpenCL: el paralelismo masivo de la GPU sí rinde

OpenCL alcanzó speedups de hasta 39,37x en la entrada más grande, gracias a que su kernel lanza un espacio de trabajo masivo (miles de millones de tareas), ideal para este problema. En la entrada más chica el speedup baja a 6,04x, lo que sugiere que el costo de transferencia de datos y compilación del kernel pesa más cuando el trabajo es reducido.

Reflexión: dificultades y lecciones aprendidas

La principal dificultad del proyecto no fue implementar las versiones secuencial, Pthreads, MPI y OpenCL, sino comprender por qué el rendimiento obtenido era muy diferente al esperado. Aunque la teoría indicaba que el paralelismo debía ofrecer mejoras importantes, las pruebas demostraron que el costo asociado a la gestión de hilos, procesos y memoria reducía considerablemente esos beneficios.

En Pthreads se identificó que el diseño de la estructura compartida provocaba un uso poco eficiente de la memoria caché, generando un cuello de botella que solo fue evidente al analizar el comportamiento del programa bajo carga. En MPI, el principal inconveniente fue el costo de comunicación entre procesos, ya que todos recibían la información completa antes de iniciar el trabajo, haciendo que el tiempo de intercambio superara las ventajas de paralelizar el procesamiento.

También se encontró que la generación de datos utilizaba `rand()` sin inicializar una semilla con `srand()`, por lo que todas las ejecuciones producían los mismos datos. Aunque esto permitió

realizar pruebas consistentes, limitó la posibilidad de evaluar distintos escenarios sin modificar el código. En OpenCL, por otra parte, fue necesario utilizar operaciones atómicas para evitar conflictos cuando varios *work-items* intentaban actualizar el mismo resultado simultáneamente.

Finalmente, las herramientas de análisis presentaron limitaciones para medir correctamente el rendimiento de las versiones paralelas, especialmente en OpenCL. Esto hizo necesario complementar la evaluación con el análisis de los resultados. De antemano ya existían señales de que el costo del paralelismo podía superar sus beneficios en ciertos casos, algo que terminó confirmándose durante las pruebas finales.